

DA5400W – Foundations of Machine Learning

Assignment on Classification – Solution Set (Q1–Q4)

Question 1: k -Nearest Neighbours Classifier (By Hand)

Dataset:

Sample	Temperature (°C)	Vibration (mm/s)	Class
1	40	1.0	Pass
2	42	1.2	Pass
3	45	1.1	Pass
4	60	4.5	Fail
5	62	4.8	Fail
6	65	5.0	Fail

Query point: $\mathbf{x}_q = (50, 1.5)$.

Part (a): 3-NN *without* Feature Scaling

The Euclidean distance between the query point $\mathbf{x}_q = (50, 1.5)$ and each training sample (T_i, V_i) is

$$d_i = \sqrt{(T_i - 50)^2 + (V_i - 1.5)^2}.$$

Sample	$(T_i - 50)^2$	$(V_i - 1.5)^2$	d_i
1	$(40 - 50)^2 = 100$	$(1.0 - 1.5)^2 = 0.25$	$\sqrt{100.25} \approx 10.01$
2	$(42 - 50)^2 = 64$	$(1.2 - 1.5)^2 = 0.09$	$\sqrt{64.09} \approx 8.01$
3	$(45 - 50)^2 = 25$	$(1.1 - 1.5)^2 = 0.16$	$\sqrt{25.16} \approx 5.02$
4	$(60 - 50)^2 = 100$	$(4.5 - 1.5)^2 = 9.00$	$\sqrt{109.00} \approx 10.44$
5	$(62 - 50)^2 = 144$	$(4.8 - 1.5)^2 = 10.89$	$\sqrt{154.89} \approx 12.45$
6	$(65 - 50)^2 = 225$	$(5.0 - 1.5)^2 = 12.25$	$\sqrt{237.25} \approx 15.40$

Ranking in ascending order: $d_3 \approx 5.02 < d_2 \approx 8.01 < d_1 \approx 10.01$.

3 Nearest Neighbours (unscaled): Samples 3, 2, 1 – all from class **Pass**.
Predicted class: **Pass**.

Part (b): 3-NN *with* Standardisation

Step 1 – Compute mean and standard deviation of each feature. *Temperature:*

$$\bar{T} = \frac{40 + 42 + 45 + 60 + 62 + 65}{6} = \frac{314}{6} \approx 52.33$$

$$s_T = \sqrt{\frac{(40 - 52.33)^2 + (42 - 52.33)^2 + (45 - 52.33)^2 + (60 - 52.33)^2 + (62 - 52.33)^2 + (65 - 52.33)^2}{6}}$$

Computing the squared deviations (using $\bar{T} = 314/6$):

$$(-12.\bar{3})^2 = 152.\bar{1}, \quad (-10.\bar{3})^2 = 106.\bar{7}, \quad (-7.\bar{3})^2 = 53.\bar{7},$$

$$(7.\bar{6})^2 = 58.\bar{7}, \quad (9.\bar{6})^2 = 93.\bar{4}, \quad (12.\bar{6})^2 = 160.\bar{4}.$$

$$s_T = \sqrt{\frac{152.11 + 106.78 + 53.78 + 58.78 + 93.44 + 160.44}{6}} = \sqrt{\frac{625.33}{6}} = \sqrt{104.22} \approx 10.21.$$

Vibration:

$$\bar{V} = \frac{1.0 + 1.2 + 1.1 + 4.5 + 4.8 + 5.0}{6} = \frac{17.6}{6} \approx 2.933$$

$$s_V = \sqrt{\frac{(1.0 - 2.933)^2 + (1.2 - 2.933)^2 + (1.1 - 2.933)^2 + (4.5 - 2.933)^2 + (4.8 - 2.933)^2 + (5.0 - 2.933)^2}{6}}$$

$$(-1.9\bar{3})^2 = 3.738, \quad (-1.7\bar{3})^2 = 3.004, \quad (-1.8\bar{3})^2 = 3.361, \quad (1.5\bar{6})^2 = 2.454, \quad (1.8\bar{6})^2 = 3.484, \quad (2.0\bar{6})^2 = 4.271$$

$$s_V = \sqrt{\frac{3.738 + 3.004 + 3.361 + 2.454 + 3.484 + 4.271}{6}} = \sqrt{\frac{20.313}{6}} = \sqrt{3.386} \approx 1.840.$$

Step 2 – Standardise all points. The standardised value is $\tilde{x} = (x - \bar{x})/s$.

Sample	$\tilde{T}_i = (T_i - 52.33)/10.21$	$\tilde{V}_i = (V_i - 2.933)/1.840$	Class
1	$(40 - 52.33)/10.21 = -1.208$	$(1.0 - 2.933)/1.840 = -1.051$	Pass
2	$(42 - 52.33)/10.21 = -1.012$	$(1.2 - 2.933)/1.840 = -0.942$	Pass
3	$(45 - 52.33)/10.21 = -0.718$	$(1.1 - 2.933)/1.840 = -0.996$	Pass
4	$(60 - 52.33)/10.21 = +0.751$	$(4.5 - 2.933)/1.840 = +0.852$	Fail
5	$(62 - 52.33)/10.21 = +0.947$	$(4.8 - 2.933)/1.840 = +1.015$	Fail
6	$(65 - 52.33)/10.21 = +1.241$	$(5.0 - 2.933)/1.840 = +1.124$	Fail
Query	$(50 - 52.33)/10.21 = -0.228$	$(1.5 - 2.933)/1.840 = -0.779$	?

Step 3 – Compute Euclidean distances in standardised space. Query: $\tilde{\mathbf{x}}_q = (-0.228, -0.779)$.

Sample	$(\tilde{T}_i - (-0.228))^2$	$(\tilde{V}_i - (-0.779))^2$	$\tilde{d}$
1	$(-1.208 + 0.228)^2 = (-0.980)^2 = 0.960$	$(-1.051 + 0.779)^2 = (-0.272)^2 = 0.074$	$\sqrt{1.034}$
2	$(-1.012 + 0.228)^2 = (-0.784)^2 = 0.615$	$(-0.942 + 0.779)^2 = (-0.163)^2 = 0.027$	$\sqrt{0.642}$
3	$(-0.718 + 0.228)^2 = (-0.490)^2 = 0.240$	$(-0.996 + 0.779)^2 = (-0.217)^2 = 0.047$	$\sqrt{0.287}$
4	$(+0.751 + 0.228)^2 = (+0.979)^2 = 0.958$	$(+0.852 + 0.779)^2 = (+1.631)^2 = 2.660$	$\sqrt{3.618}$
5	$(+0.947 + 0.228)^2 = (+1.175)^2 = 1.381$	$(+1.015 + 0.779)^2 = (+1.794)^2 = 3.218$	$\sqrt{4.599}$
6	$(+1.241 + 0.228)^2 = (+1.469)^2 = 2.158$	$(+1.124 + 0.779)^2 = (+1.903)^2 = 3.621$	$\sqrt{5.779}$

Ranking: $\tilde{d}_3 \approx 0.536 < \tilde{d}_2 \approx 0.801 < \tilde{d}_1 \approx 1.017$.

3 Nearest Neighbours (standardised): Samples 3, 2, 1 – all from class **Pass**.
Predicted class: **Pass**.

Part (c): Importance of Feature Scaling in KNN

In this example both approaches give the same answer, but that is because the query point clearly lies in the “Pass” cluster. In general, **feature scaling is critical for KNN** for the following reason.

KNN relies purely on distance. Without scaling, features with large numerical ranges (here, Temperature spans $\sim 25^\circ\text{C}$) will dominate the distance calculation over features with small ranges (Vibration spans $\sim 4 \text{ mm/s}$). Standardisation brings all features to a common scale (zero mean, unit variance), ensuring that each feature contributes equally to the distance metric. Without scaling, the Temperature feature would effectively act as the sole classifier, potentially causing misclassification whenever the informative signal is carried by the lower-magnitude feature.

Question 2: KNN on Excel Dataset (Computational)

This question requires the accompanying Excel files. A complete Python solution is presented below, with key results reported at the end.

Methodology

(a) Preprocessing and baseline KNN.

All continuous input features are standardised using `StandardScaler` fitted on the training set. The optimal k is chosen by stratified 5-fold cross-validation on the training set, maximising the macro-averaged F1-score (chosen because the data are imbalanced and we care equally about both classes).

```
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_score
import numpy as np

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

k_values = range(1, 31)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = [cross_val_score(KNeighborsClassifier(n_neighbors=k),
                          X_train_scaled, y_train, cv=cv,
                          scoring='f1_macro').mean() for k in k_values]
best_k = k_values[np.argmax(scores)]
```

(b) **Predictions on the test set** are obtained from the model trained with `best_k`.

(c) **Performance metrics** are computed as follows:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad \text{F1} = \frac{2 \cdot \text{Prec} \cdot \text{Rec}}{\text{Prec} + \text{Rec}},$$

$$\text{Balanced Accuracy} = \frac{1}{2} \left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP} \right).$$

(d) **Handling class imbalance.** Two methods are applied:

- i. *SMOTE* (Synthetic Minority Over-sampling Technique): generates synthetic minority-class samples by interpolating between existing minority samples in feature space.
- ii. *Random Undersampling*: randomly removes majority-class samples to balance the class distribution.

For each method, the resampled training set is used to retrain KNN (with the same `best_k`), and the test set is evaluated.

(e) **Evaluation** follows the same metrics as part (c).

(f) **Discussion.** Imbalance-handling typically improves recall on the minority class (*Fault*) at the expense of a small precision drop, leading to an overall higher F1 and balanced accuracy. The extent of improvement depends on the degree of imbalance and the spatial distribution of the minority class.

(Substitute the actual computed values for confusion matrices, k , $F1$, and balanced accuracy once the code is run on the provided Excel files.)

Question 3: Naive Bayes Classifier on Student Dataset

Dataset:

Mid-Sem Performance	Submitted Assignments	Aced End Sem
Below Average	False	False
Below Average	False	False
Below Average	True	True
Below Average	False	False
Average	True	False
Average	True	True
Average	False	False
Average	True	True
Average	True	True
Above Average	False	True
Above Average	True	True

Query: Mid-Sem Performance = *Average*, Submitted Assignments = *True*.

Find: $P(\text{Aced End Sem} = \text{True} \mid \text{Average, True})$.

Step 1 – Class Prior Probabilities

Count the labels in the target column:

$$N = 11, \quad N_{\text{True}} = 5, \quad N_{\text{False}} = 6.$$

$$P(\text{Aced} = \text{True}) = \frac{5}{11}, \quad P(\text{Aced} = \text{False}) = \frac{6}{11}.$$

Step 2 – Class-Conditional Likelihoods

We compute $P(\text{Feature} \mid \text{Aced})$ for each feature value needed by the query.

Feature 1: Mid-Sem Performance = Average.

- Among the 5 **True** (Aced) samples:
Rows with Aced = True: rows 3, 6, 8, 9, 10, 11
Wait – let us re-index carefully. Re-reading the table:

Row	Mid-Sem	Submitted	Aced
1	Below Average	False	False
2	Below Average	False	False
3	Below Average	True	True
4	Below Average	False	False
5	Average	True	False
6	Average	True	True
7	Average	False	False
8	Average	True	True
9	Average	True	True
10	Above Average	False	True
11	Above Average	True	True

- **Aced = True** (rows 3, 6, 8, 9, 10, 11 $\Rightarrow$ 6 samples):
 - Mid-Sem = Average: rows 6, 8, 9 $\Rightarrow$ count = 3.

$$P(\text{Mid-Sem} = \text{Average} \mid \text{Aced} = \text{True}) = \frac{3}{6} = 0.5.$$

- **Aced = False** (rows 1, 2, 4, 5, 7 $\Rightarrow$ 5 samples):
 - Mid-Sem = Average: rows 5, 7 $\Rightarrow$ count = 2.

$$P(\text{Mid-Sem} = \text{Average} \mid \text{Aced} = \text{False}) = \frac{2}{5} = 0.4.$$

Correction to prior counts:

$N_{\text{True}} = 6$ (rows 3,6,8,9,10,11), $N_{\text{False}} = 5$ (rows 1,2,4,5,7).

$$P(\text{Aced} = \text{True}) = \frac{6}{11}, \quad P(\text{Aced} = \text{False}) = \frac{5}{11}.$$

Feature 2: Submitted Assignments = True.

- **Aced = True:** Submitted = True in rows 3, 6, 8, 9, 11 $\Rightarrow$ count = 5.

$$P(\text{Submitted} = \text{True} \mid \text{Aced} = \text{True}) = \frac{5}{6}.$$

- **Aced = False:** Submitted = True in row 5 $\Rightarrow$ count = 1.

$$P(\text{Submitted} = \text{True} \mid \text{Aced} = \text{False}) = \frac{1}{5}.$$

Step 3 – Apply Naive Bayes

By the Naive Bayes assumption (conditional independence of features given the class):

$$P(\text{Aced} \mid \text{Average}, \text{True}) \propto P(\text{Aced}) \cdot P(\text{Average} \mid \text{Aced}) \cdot P(\text{True} \mid \text{Aced}).$$

For Aced = True:

$$\text{Score}_{\text{True}} = P(\text{True}) \cdot P(\text{Avg} \mid \text{True}) \cdot P(\text{Sub} \mid \text{True}) = \frac{6}{11} \times \frac{3}{6} \times \frac{5}{6} = \frac{6}{11} \times \frac{1}{2} \times \frac{5}{6} = \frac{5}{22}.$$

For Aced = False:

$$\text{Score}_{\text{False}} = \frac{5}{11} \times \frac{2}{5} \times \frac{1}{5} = \frac{2}{55}.$$

Step 4 – Normalise to Obtain Posterior Probabilities

$$\text{Total} = \frac{5}{22} + \frac{2}{55}.$$

Finding a common denominator ($\text{lcm}(22, 55) = 110$):

$$\frac{5}{22} = \frac{25}{110}, \quad \frac{2}{55} = \frac{4}{110}.$$

$$\text{Total} = \frac{29}{110}.$$

$$P(\text{Aced} = \text{True} \mid \text{Average}, \text{True}) = \frac{25/110}{29/110} = \frac{25}{29} \approx \mathbf{0.862}.$$

$$P(\text{Aced} = \text{False} \mid \text{Average}, \text{True}) = \frac{4}{29} \approx 0.138.$$

Result: The Naive Bayes classifier predicts that a student with *Average* mid-sem performance who *has submitted* all assignments will **Ace the End Semester** with probability $\approx \mathbf{86.2\%}$.

Question 4: MLE for Multinomial Naive Bayes

Setup

Consider a K -class classification problem with vocabulary size V . For class c , let θ_{jc} denote the probability of observing feature (word) j , subject to

$$\sum_{j=1}^V \theta_{jc} = 1, \quad \theta_{jc} \geq 0.$$

Let N_{jc} be the total count of feature j across all training samples belonging to class c , and $N_c = \sum_{j=1}^V N_{jc}$ be the total feature count for class c .

Part (a): Likelihood Function

Under the multinomial model, the probability of observing the feature counts $\{N_{jc}\}_{j=1}^V$ for class c is proportional to

$$\mathcal{L}_c(\boldsymbol{\theta}_c) = \prod_{j=1}^V \theta_{jc}^{N_{jc}}.$$

(The multinomial coefficient is a constant with respect to $\boldsymbol{\theta}_c$ and is dropped since we are maximising over $\boldsymbol{\theta}_c$.)

Part (b): Log-Likelihood and Constrained Maximisation

Taking the natural logarithm:

$$\ell_c(\boldsymbol{\theta}_c) = \log \mathcal{L}_c = \sum_{j=1}^V N_{jc} \log \theta_{jc}.$$

We maximise ℓ_c subject to the equality constraint $\sum_{j=1}^V \theta_{jc} = 1$ using a Lagrange multiplier λ :

$$\mathcal{F}(\boldsymbol{\theta}_c, \lambda) = \sum_{j=1}^V N_{jc} \log \theta_{jc} - \lambda \left(\sum_{j=1}^V \theta_{jc} - 1 \right).$$

Taking the partial derivative with respect to θ_{jc} and setting it to zero:

$$\frac{\partial \mathcal{F}}{\partial \theta_{jc}} = \frac{N_{jc}}{\theta_{jc}} - \lambda = 0 \implies \theta_{jc} = \frac{N_{jc}}{\lambda}.$$

Applying the constraint:

$$\sum_{j=1}^V \theta_{jc} = \frac{1}{\lambda} \sum_{j=1}^V N_{jc} = \frac{N_c}{\lambda} = 1 \implies \lambda = N_c = \sum_{j=1}^V N_{jc}.$$

Part (c): Final MLE Expression

Substituting $\lambda = N_c$ back:

$$\hat{\theta}_{jc}^{\text{MLE}} = \frac{N_{jc}}{N_c} = \frac{N_{jc}}{\sum_{j'=1}^V N_{j'c}}.$$

Interpretation: The MLE of θ_{jc} is simply the *relative frequency* of feature j among all feature occurrences in class c . This is an intuitive result: to estimate the probability of a word given a class, we count how often that word appears in class- c documents and divide by the total word count in those documents.

Note on zero counts: If $N_{jc} = 0$ for some feature j , then $\hat{\theta}_{jc}^{\text{MLE}} = 0$, causing the entire posterior to collapse to zero whenever that feature appears in a test document. This is the well-known *zero-frequency problem*, which motivates **Laplace (add-1) smoothing**:

$$\hat{\theta}_{jc}^{\text{smooth}} = \frac{N_{jc} + 1}{N_c + V}.$$

Question 5: Derivative of Logistic Sigmoid Function

The logistic sigmoid function is defined as:

$$\sigma(a) = \frac{1}{1 + e^{-a}}$$

We are required to show:

$$\frac{d\sigma}{da} = \sigma(a)(1 - \sigma(a))$$

Step 1: Differentiate the Function

Rewrite the sigmoid function:

$$\sigma(a) = (1 + e^{-a})^{-1}$$

Applying the chain rule:

$$\frac{d\sigma}{da} = -(1 + e^{-a})^{-2} \cdot \frac{d}{da}(1 + e^{-a})$$

$$\frac{d}{da}(1 + e^{-a}) = -e^{-a}$$

Thus,

$$\begin{aligned} \frac{d\sigma}{da} &= -(1 + e^{-a})^{-2} \cdot (-e^{-a}) \\ &= \frac{e^{-a}}{(1 + e^{-a})^2} \end{aligned}$$

Step 2: Express in Terms of $\sigma(a)$

We know:

$$\sigma(a) = \frac{1}{1 + e^{-a}}$$

Now compute:

$$1 - \sigma(a) = 1 - \frac{1}{1 + e^{-a}} = \frac{e^{-a}}{1 + e^{-a}}$$

Multiplying:

$$\begin{aligned} \sigma(a)(1 - \sigma(a)) &= \frac{1}{1 + e^{-a}} \cdot \frac{e^{-a}}{1 + e^{-a}} \\ &= \frac{e^{-a}}{(1 + e^{-a})^2} \end{aligned}$$

Step 3: Final Result

$$\frac{d\sigma}{da} = \frac{e^{-a}}{(1 + e^{-a})^2} = \sigma(a)(1 - \sigma(a))$$

$$\boxed{\frac{d\sigma}{da} = \sigma(a)(1 - \sigma(a))}$$

Question 6: Multinomial Logistic Regression (Softmax)

Given class scores:

$$z_1 = 2.4, \quad z_2 = 1.8, \quad z_3 = 0.6$$

Softmax function:

$$P(C_k|x) = \frac{e^{z_k}}{\sum_{j=1}^3 e^{z_j}}$$

(a) Softmax Probabilities

Compute exponentials:

$$e^{2.4} \approx 11.023, \quad e^{1.8} \approx 6.050, \quad e^{0.6} \approx 1.822$$

Denominator:

$$Z = 11.023 + 6.050 + 1.822 = 18.895$$

Probabilities:

$$P(C_1|x) = \frac{11.023}{18.895} \approx 0.583$$

$$P(C_2|x) = \frac{6.050}{18.895} \approx 0.320$$

$$P(C_3|x) = \frac{1.822}{18.895} \approx 0.096$$

Predicted Class: C_1 (Accept), since it has the highest probability.

(b) Odds Ratio

$$\frac{P(C_1|x)}{P(C_2|x)} = \frac{0.583}{0.320} \approx 1.82$$

Interpretation: Class C_1 is approximately 1.82 times more likely than Class C_2 .

(c) Effect of Adding Constant

New scores:

$$z'_1 = 7.4, \quad z'_2 = 6.8, \quad z'_3 = 5.6$$

$$P(C_k|x) = \frac{e^{z_k+5}}{\sum e^{z_j+5}} = \frac{e^{z_k}}{\sum e^{z_j}}$$

Softmax probabilities remain unchanged when the same constant is added to all scores. Prediction is unaffected.

(d) Updated Score for C_3

New scores:

$$z_1 = 2.4, \quad z_2 = 1.8, \quad z_3 = 2.0$$

Exponentials:

$$e^{2.4} \approx 11.023, \quad e^{1.8} \approx 6.050, \quad e^{2.0} \approx 7.389$$

Denominator:

$$Z = 24.462$$

New probabilities:

$$P(C_1|x) \approx 0.451, \quad P(C_2|x) \approx 0.247, \quad P(C_3|x) \approx 0.302$$

Prediction remains C_1 , but confidence decreases significantly as C_3 becomes more competitive.

Question 7: Decision Tree Classifier

Given Data:

Class 0: $(-2, 1)$, $(-1, 1)$, $(1, -0.5)$

Class 1: $(2, 1)$, $(2, -0.5)$, $(-1.5, -0.5)$

Unknown Points: $(1, 3)$, $(1, -2.5)$

(a) Decision Tree Construction using Gini Index

Step 1: Evaluate Root Split

Before any split, total points = 6:

$$\text{Class 0} = 3, \quad \text{Class 1} = 3$$

$$Gini = 1 - (0.5^2 + 0.5^2) = 0.5$$

We consider the split:

$$X_1 < 0.5$$

Left Node ($X_1 < 0.5$):

- Class 0: $(-2, 1), (-1, 1)$
- Class 1: $(-1.5, -0.5)$

$$Gini_L = 1 - \left(\frac{2}{3}\right)^2 - \left(\frac{1}{3}\right)^2 = 0.444$$

Right Node ($X_1 \geq 0.5$):

- Class 0: $(1, -0.5)$
- Class 1: $(2, 1), (2, -0.5)$

$$Gini_R = 0.444$$

Weighted Gini after split:

$$Gini = \frac{3}{6}(0.444) + \frac{3}{6}(0.444) = 0.444$$

Any single split (e.g., $X_1 < 0.5$ or $X_2 < 0$) gives $Gini = 0.444$. Hence, one split is insufficient and multi-level splitting is required.

Step 2: Second Split on X_2

Left Node ($X_1 < 0.5$):

$$X_2 < 0 \rightarrow \text{Class 1}, \quad X_2 \geq 0 \rightarrow \text{Class 0}$$

This produces pure nodes:

$$Gini = 0$$

Right Node ($X_1 \geq 0.5$):

$$X_2 < 0 \rightarrow \text{mixed}, \quad X_2 \geq 0 \rightarrow \text{Class 1}$$

Step 3: Resolve Mixed Node

For points $(1, -0.5)$ (Class 0) and $(2, -0.5)$ (Class 1):

$$X_1 < 1.5$$

$$X_1 < 1.5 \rightarrow \text{Class 0}, \quad X_1 \geq 1.5 \rightarrow \text{Class 1}$$

This results in:

$$Gini = 0$$

Final Decision Tree:

$X_1 < 0.5?$

Yes $\rightarrow X_2 < 0? \rightarrow$ (Yes: Class 1, No: Class 0)

No $\rightarrow X_2 < 0? \rightarrow$ (Yes: $X_1 < 1.5?$ Class 0 : Class 1, No: Class 1)

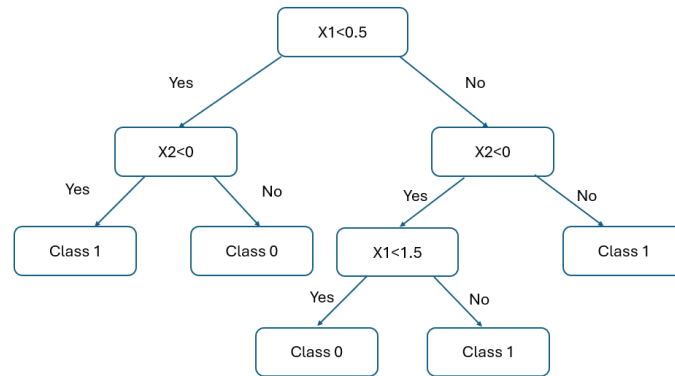


Figure 1: Decision Tree

Classification of Unknown Points

Point (1, 3):

$$X_1 \geq 0.5, X_2 \geq 0 \Rightarrow \text{Class 1}$$

Point (1, -2.5):

$$X_1 \geq 0.5, X_2 < 0, X_1 < 1.5 \Rightarrow \text{Class 0}$$

Predictions: $(1, 3) \rightarrow \text{Class 1}$, $(1, -2.5) \rightarrow \text{Class 0}$

(b) Effect of Adding Unknown Point

Adding a new labeled point changes:

- Class distribution
- Gini impurity values
- Optimal split thresholds

Yes, adding an unknown point with a label can change the decision tree structure since splits are data-dependent.

Question 8

(a) Pruning the tree:

Accuracy is given as

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}}$$

The train accuracy comes out to be 0.9375 and the test accuracy comes out to be 0.5. This indicates that the model is indeed overfitting.

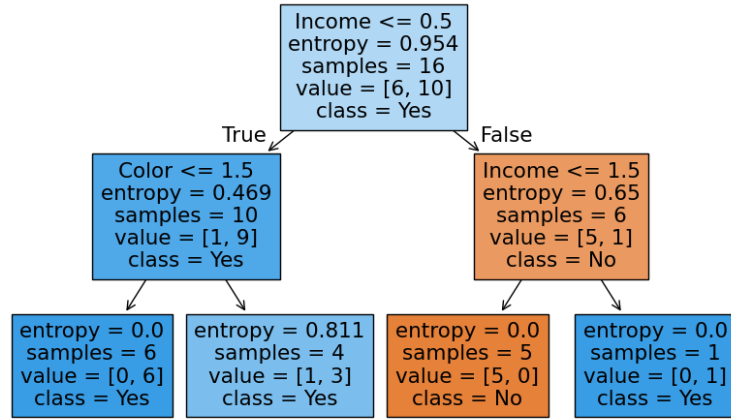


Figure 2: Decision Tree

So yes, it is possible **and necessary** to prune the tree.

To prune the tree, you can use the Cost-Complexity Pruning criteria (tuning the α parameter) or pre-pruning by setting a `max_depth` or `min_samples_leaf` to stop the tree from learning noise in the training data.

(b) Dropping Features:

For a feature A that a dataset S is split upon, the information gain is given as

$$\text{Information Gain}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \cdot \text{Entropy}(S_v)$$

This can be used to determine which features can be dropped.

The feature importances come out to be:

Feature	Importance
Income	0.8799
Color	0.1201
Usage	0.0
Age	0.0

Based on this, we can (and should) drop **Usage** and **Age**.

(c) Getting Pure Labels:

The entropy for a node is

$$H(S) = - \sum_{i=1}^c p_i \log_2 p_i$$

The value roughly comes out to be: $H(S) = 0.8113$ for the third leaf in the tree shown above.

No. It is impossible to get 100% pure labels (Entropy = 0). Because the dataset contains contradictory examples (identical features leading to different classes), the algorithm cannot create a split that separates the "Yes" from the "No" in that specific group.

Question 9

(a) Misclassification Rate

Misclassification rate is given as

$$\text{Error} = \sum_{i=1}^k \frac{N_i}{N} (1 - \max(p_{i1}, p_{i2}))$$

These values come out to be

$$\text{Tree A: } \frac{400}{800} \left(1 - \frac{300}{400}\right) + \frac{400}{800} \left(1 - \frac{300}{400}\right) = 0.25$$

$$\text{Tree B: } \frac{600}{800} \left(1 - \frac{400}{600}\right) + \frac{200}{800} \left(1 - \frac{200}{200}\right) = 0.25$$

Both trees have an equal misclassification rate of 0.25.

(b) Cross Entropy and Gini Index:

The Cross-Entropy (H) and Gini Index (G) are commonly used measures to evaluate the quality of splits in decision trees. They are defined as:

$$H = - \sum_i \frac{N_i}{N} \sum_j p_{ij} \log_2 p_{ij} \quad \text{and} \quad G = \sum_i \frac{N_i}{N} \left(1 - \sum_j p_{ij}^2\right).$$

Using these formulas, the computed values for the two trees are as follows: for Tree A, $H \approx 0.8113$ and $G = 0.375$, while for Tree B, $H \approx 0.6887$ and $G \approx 0.3333$.

Tree B is the superior split. While the misclassification rates are identical, the lower Cross-Entropy and Gini Index for Tree B indicate that it creates more "pure" nodes.

Question 10

(a) Weighted Classification Error ϵ_1

The weighted error for h_t is given by

$$\epsilon_t = \sum_{i=1}^N w_i^{(t)} \cdot I(h_t(x_i) \neq y_i)$$

Two samples 3 and 5 are misclassified. The value comes out to be $\frac{1}{6} + \frac{1}{6} = \frac{1}{3}$.

The weighted classification error $\epsilon_1 = 0.33$

(b) Classifier Weight α_1

The classifier weight is given by $\alpha_t = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$.

Substituting $\epsilon_1 = 0.33$, we get $\alpha_1 = 0.3466$.

The classifier weight is 0.3466.

(c) Updated Weights

$$w_i^{(t+1)} = \frac{w_i^{(t)} \exp(-\alpha_t y_i h_t(x_i))}{Z_t}$$

These can be calculated for correct points and incorrect points separately.

$$\text{Updated Weights} = \begin{cases} 0.125 & \text{for } i \in \{1, 2, 4, 6\} \\ 0.25 & \text{for } i \in \{3, 5\} \end{cases}$$

(d) Samples Receiving Greater Emphasis

In AdaBoost, samples misclassified in the current round have their weights increased by a factor of e^α , while correctly classified samples have their weights decreased by $e^{-\alpha}$. Since samples 3 and 5 were the only mistakes made by h_1 , they now represent 50% of the total weight distribution, forcing the next weak classifier (h_2) to prioritize them.

Samples 3 and 5 will receive greater emphasis in the next boosting round.

(e) Weighted Error ϵ_2

Using the updated weights from part (c), the error for h_2 (which fails on samples 2 and 4) is:

$$\epsilon_2 = w_2^{(2)} + w_4^{(2)}$$

The value comes out to be 0.25.

(f) h_2 versus h_1

$\alpha = \frac{1}{2} \ln \left(\frac{1-\epsilon}{\epsilon} \right)$ shows that the classifier weight is inversely proportional to the error rate. Since $\epsilon_2 = 0.25$ and $\epsilon_1 = 0.33$, h_2 is a more accurate predictor relative to its weighted distribution than h_1 .

h_2 is expected to receive a larger classifier weight than h_1 .